

VISITOR MANAGEMENT SYSTEM

UI/UX Flow, Information Architecture & System Architecture

Enterprise Design & Engineering Specification

Stack: Node.js · Express.js · Firebase (Auth · Firestore · Storage)

Roles: Visitor · Receptionist · Employee · Security Guard · Admin

Prepared for: Design & Engineering Handoff (Figma · FigJam · Lucidchart · Draw.io · Miro)

Contents

1. Overview & User Roles
2. User Flow Diagram (Master Flow)
3. Navigation Architecture
4. Information Architecture
5. Role-Based User Journeys
 - 5.1 Visitor Journey
 - 5.2 Employee Journey
 - 5.3 Receptionist Journey
 - 5.4 Security Journey
 - 5.5 Admin Journey
6. System Architecture Diagram
7. Backend Architecture (Layered)
8. Firestore Database Structure
9. API Request Flow
10. Visitor Lifecycle Diagram (State Machine)
11. Dashboard Navigation Map
12. Design System Notes for Handoff

1. Overview & User Roles

This document specifies the end-to-end user flow, information architecture, navigation structure, and system architecture for an enterprise Visitor Management System (VMS). It is intended as a design and engineering handoff reference, exportable to Figma, FigJam, Lucidchart, Draw.io, and Miro.

User Roles

Role	Description
Visitor	External guest registering for and attending a visit
Receptionist	Front-desk operator managing walk-ins, verification, and badges
Employee	Internal host who approves/rejects visit requests
Security Guard	Verifies QR passes, manages check-in/out, monitors live visitors
Admin	Manages users, rooms, settings, and system-wide reports

Application Modules

- Authentication & Authorization
- Visitor Registration
- Employee Approval
- Reception Management
- QR Check-In / Check-Out
- Meeting Room Management
- Notifications
- Security Logs
- Reports & Analytics
- User & Role Management
- System Settings

2. User Flow Diagram (Master Flow)

The master flow shows how every role enters the system through a single authentication gate and is routed to a role-specific dashboard.

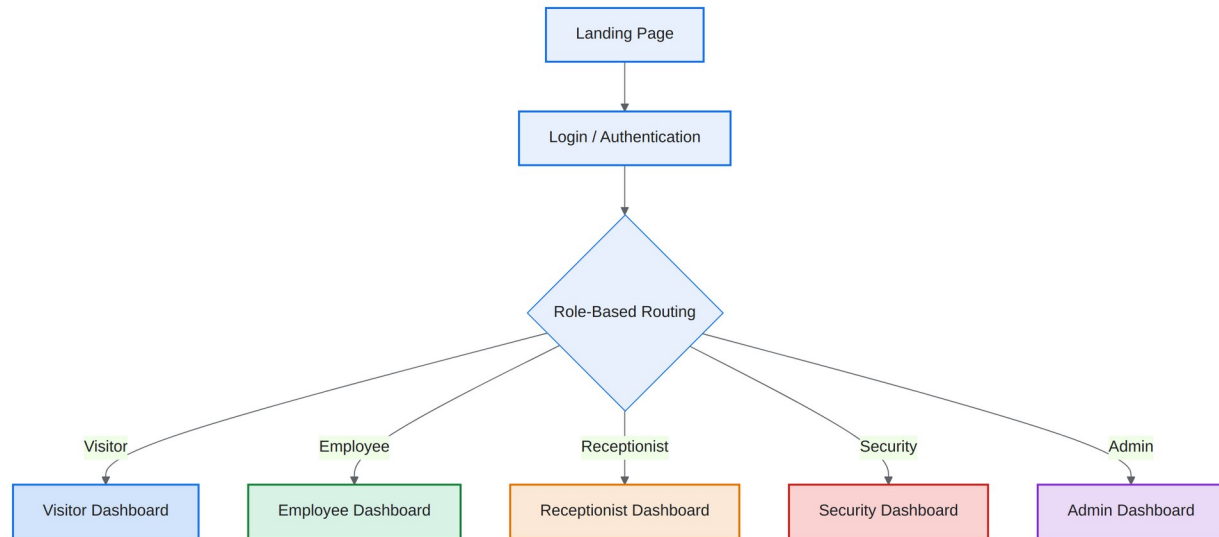


Figure 1 — Master user flow: Landing → Login → Role-Based Dashboard Routing

3. Navigation Architecture

The navigation bar is role-aware: each user sees only the modules relevant to their permissions, rendered from a shared app shell.

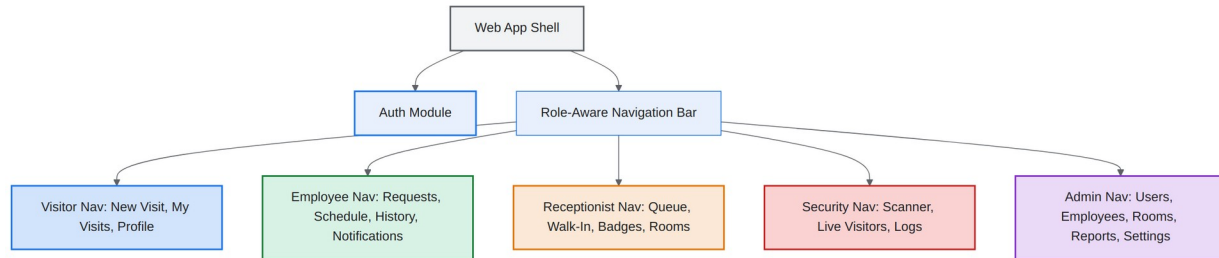


Figure 2 — Role-aware navigation architecture

4. Information Architecture

All system capabilities are organized into 11 core modules. Visitor Registration, Meeting Room Management, and Reports & Analytics further break down into sub-components shown below.

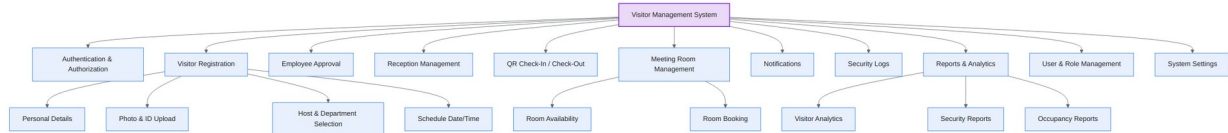


Figure 3 — Full information architecture across all modules

5. Role-Based User Journeys

5.1 Visitor Journey — Part 1: Request & Approval

The visitor journey begins on the public landing page and proceeds through registration, document upload, host selection, and scheduling before entering the approval queue.

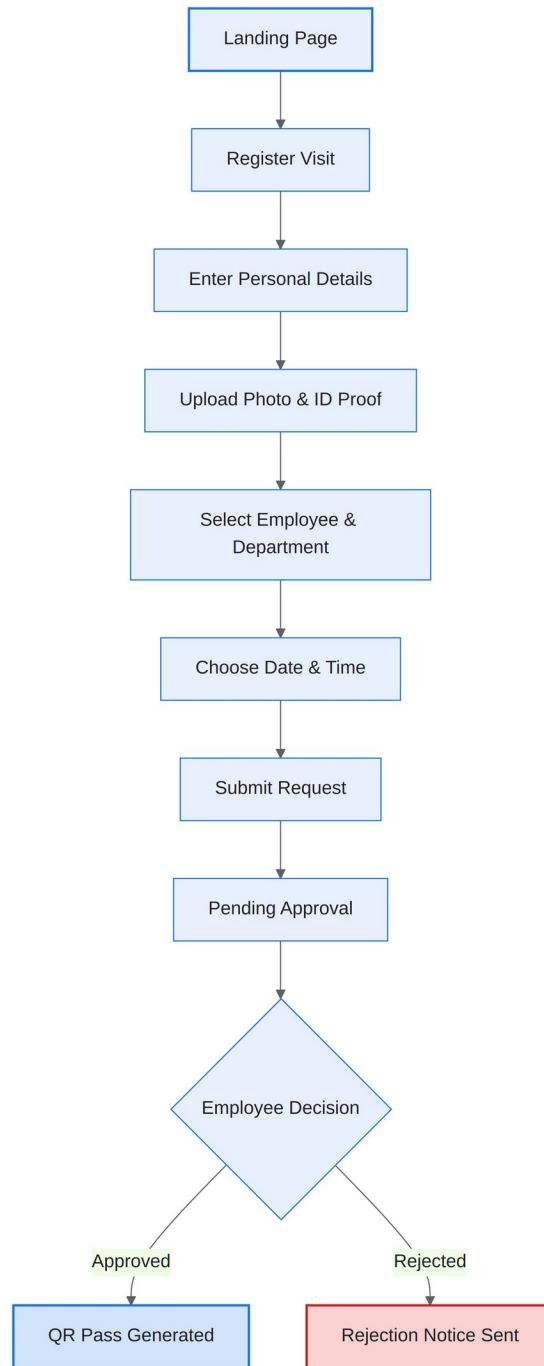


Figure 4a — Visitor journey, Part 1 (Landing Page → Employee Decision)

Part 2: Pass, Verification & Check-In

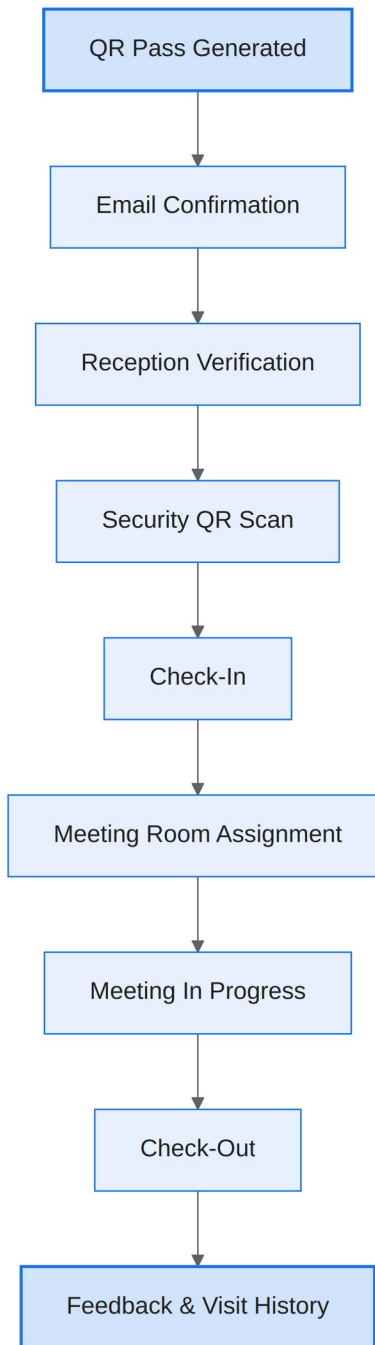


Figure 4b — Visitor journey, Part 2 (QR Pass Generated → Check-In)

5.2 Employee Journey

Employees review incoming visit requests, approve or reject them, and track their meeting schedule and visitor history.

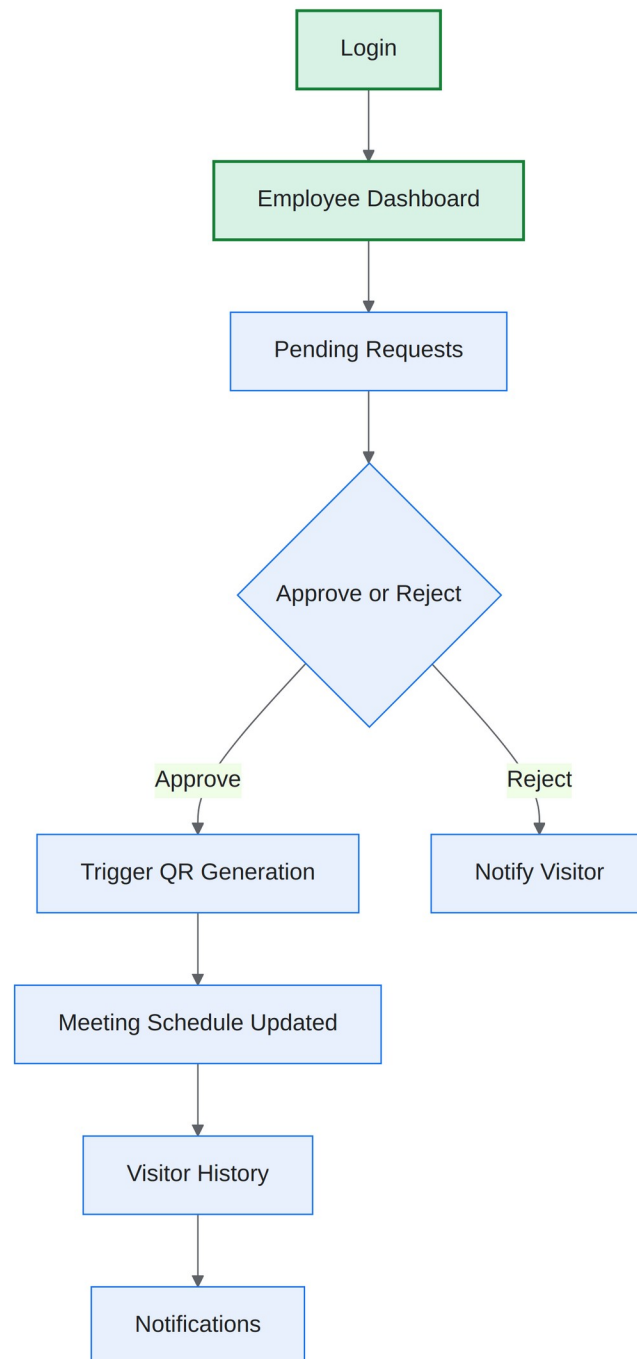


Figure 5 — Employee approval and scheduling flow

5.3 Receptionist Journey

Receptionists manage both pre-registered (expected) and walk-in visitors, verifying documents and issuing badges before notifying security.

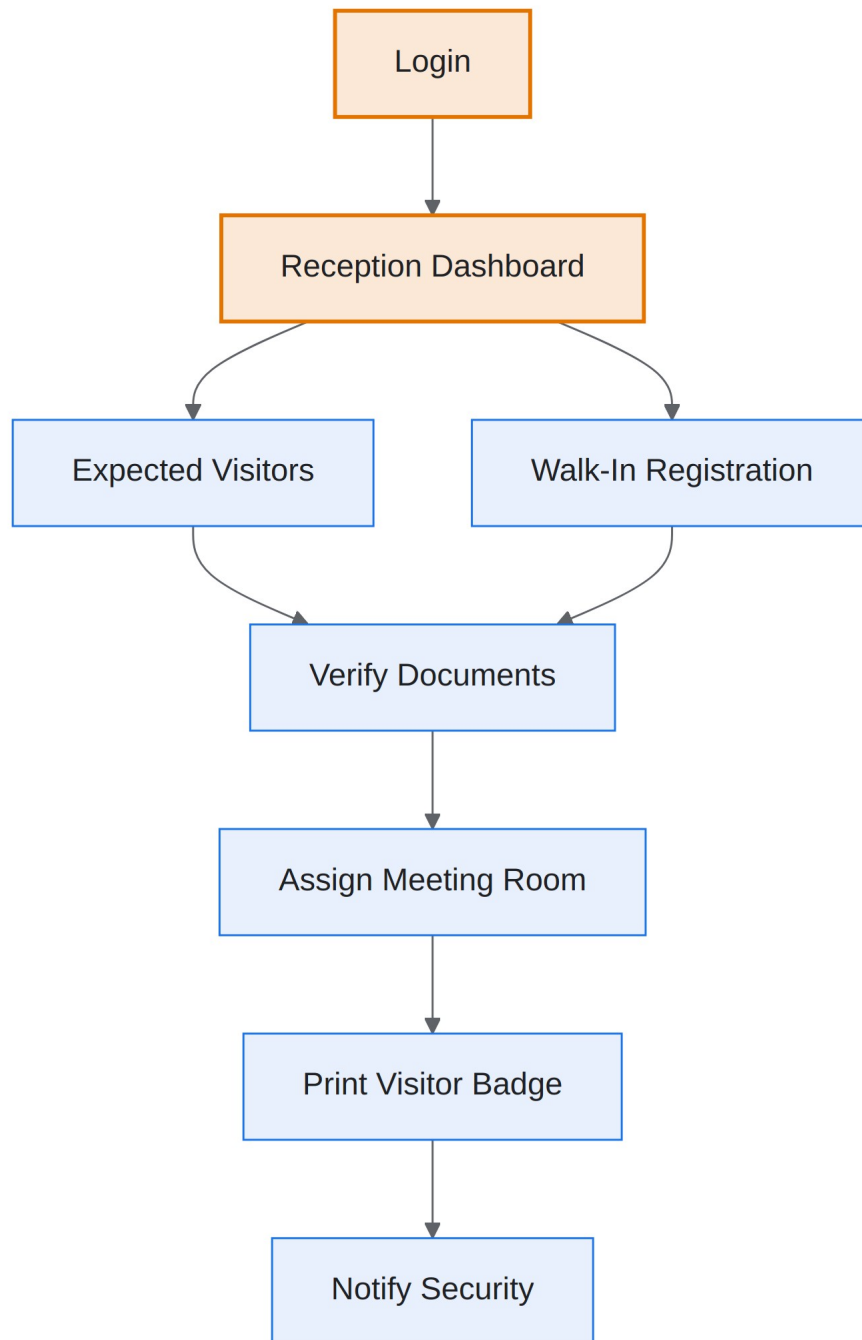


Figure 6 — Receptionist front-desk workflow

5.4 Security Journey

Security guards scan QR passes to verify identity, manage check-in/out, and maintain a live view of on-site visitors.

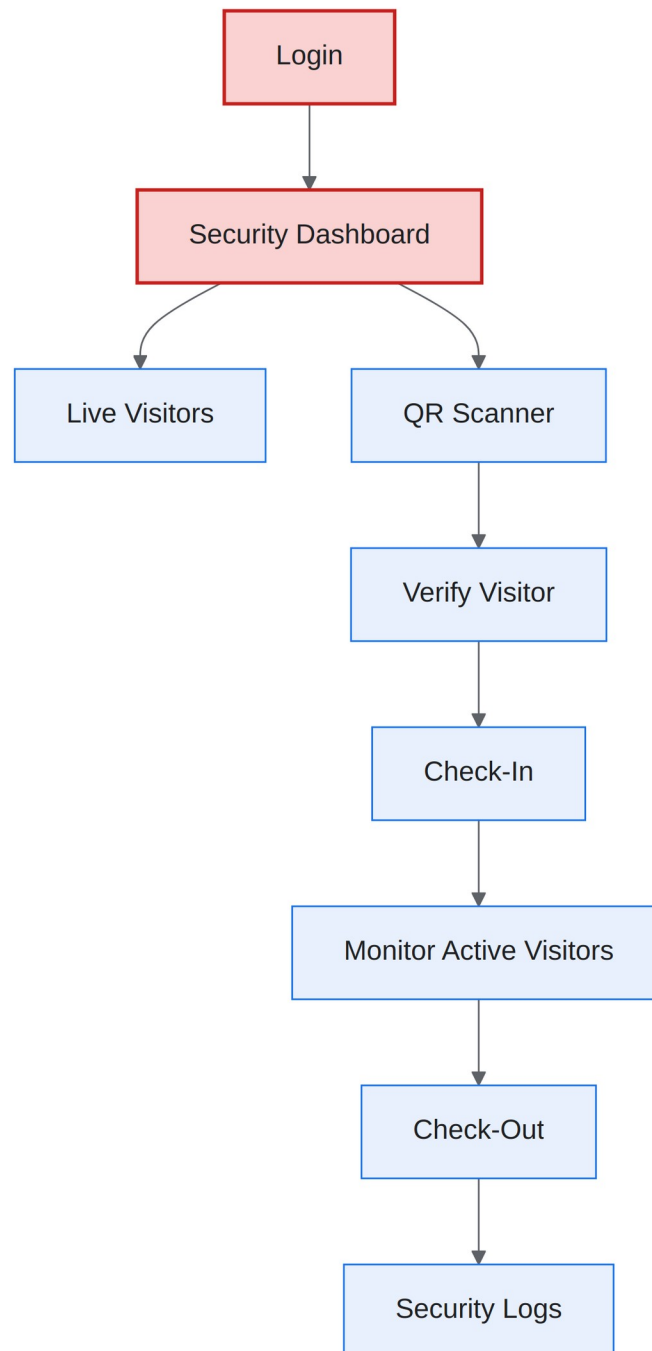


Figure 7 — Security check-in/out and monitoring flow

5.5 Admin Journey

Admins have full oversight: user/employee management, room configuration, reporting, and system settings.

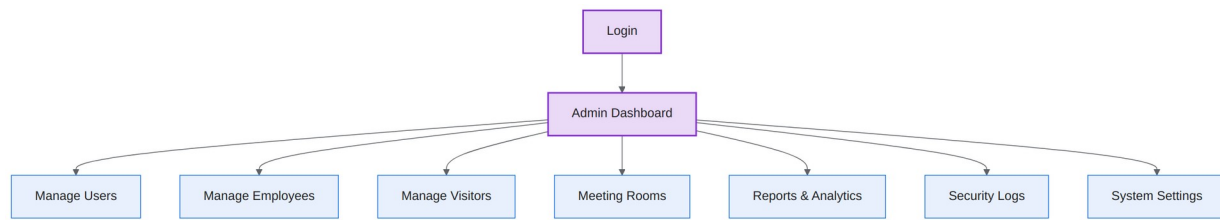


Figure 8 — Admin management and configuration flow

6. System Architecture Diagram

The system follows a layered architecture: a responsive client communicates with an Express.js REST API, which enforces authentication before invoking business logic that talks to Firebase services.

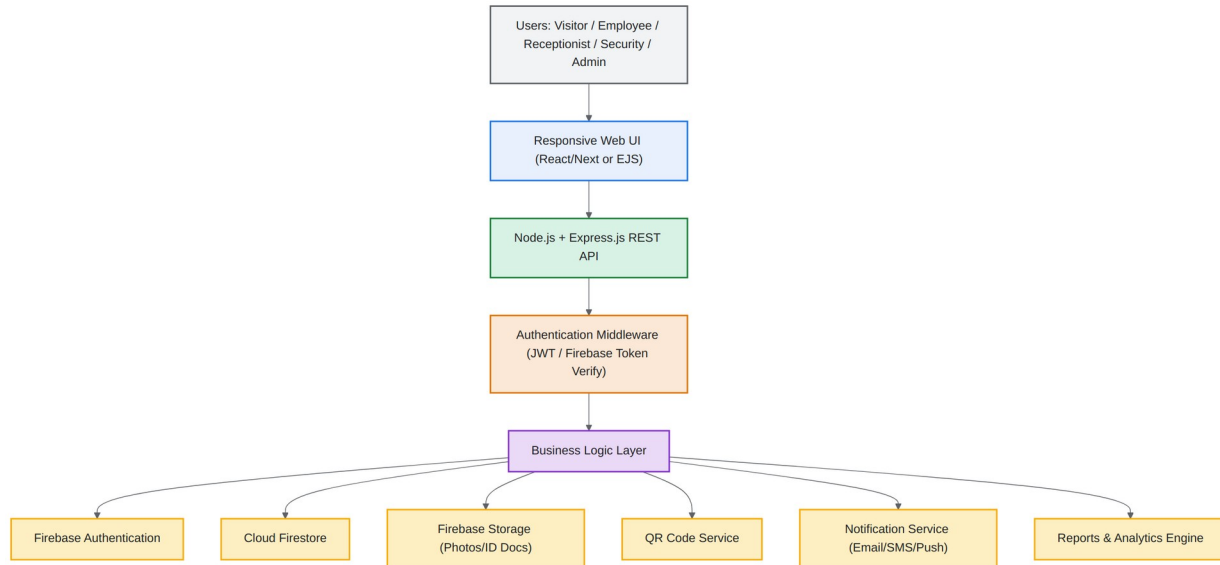


Figure 9 — End-to-end system architecture

7. Backend Architecture (Layered)

Requests flow through Express routes, controllers, middleware, and a service layer before reaching the Firestore/Storage data access layer. Logging, error handling, and caching are cross-cutting concerns applied at the controller and service layers.

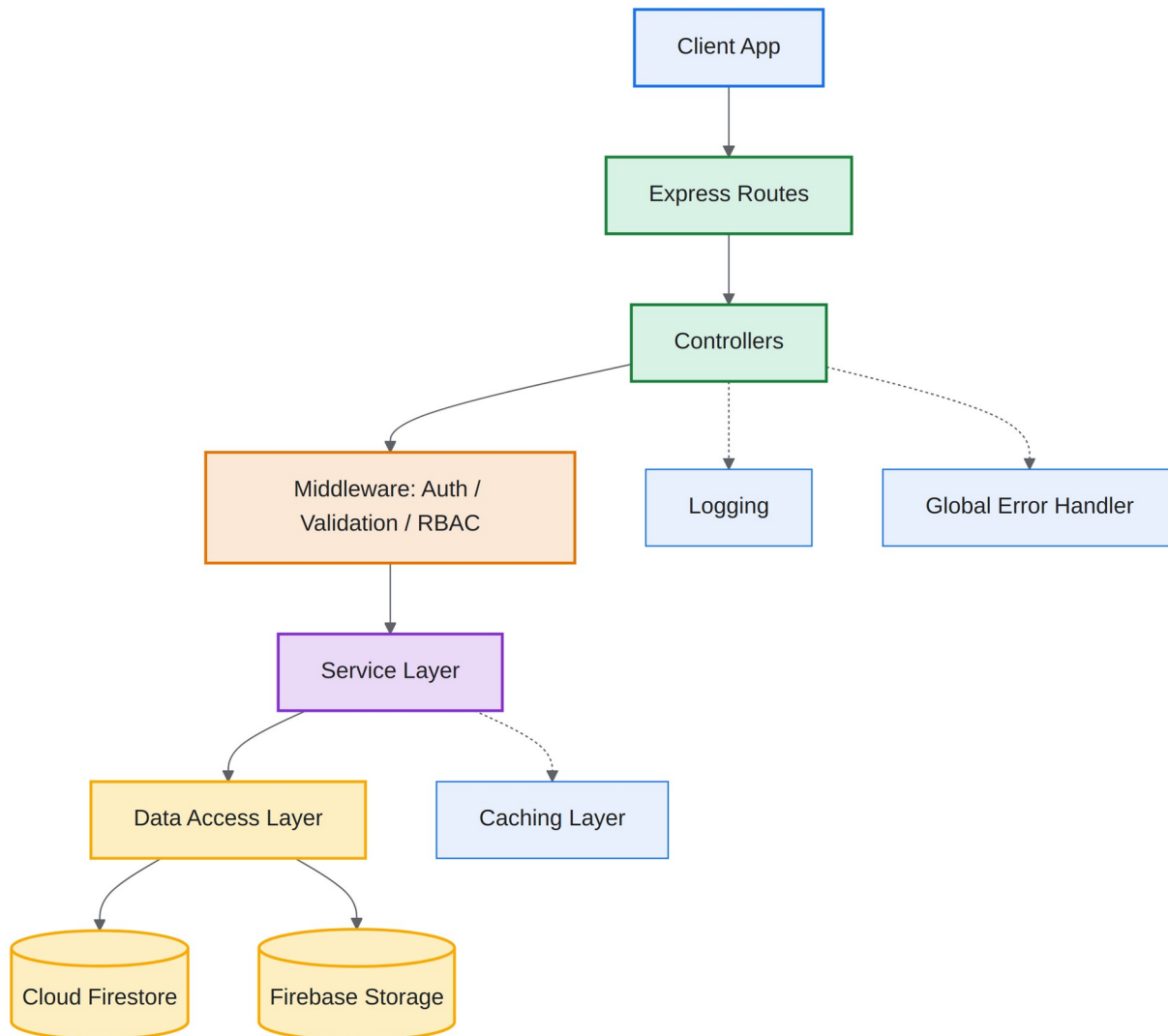


Figure 10 — Layered backend architecture with cross-cutting concerns

Suggested Folder Structure

```

/src    /routes      -> visitor.routes.js, employee.routes.js, admin.routes.js
/controllers  -> visitor.controller.js, approval.controller.js  /middleware  ->
auth.middleware.js, rbac.middleware.js, validate.middleware.js  /services      ->
qr.service.js, notification.service.js, report.service.js  /repositories ->
firestore.repository.js  /config      -> firebase.config.js, env.config.js  /utils
-> logger.js, errorHandler.js  server.js
  
```

8. Firestore Database Structure

Cloud Firestore is organized into 11 top-level collections. The diagram shows the collection hierarchy; the table below lists the key fields of each document schema.

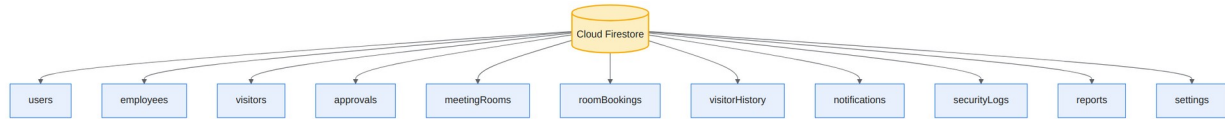


Figure 11 — Firestore collection map

Document Schemas

Collection	Key Fields
users	uid, name, email, role, phone, createdAt, status
employees	uid, name, department, designation, officeLocation
visitors	visitorId, name, photoUrl, idProofUrl, phone, email, purpose
approvals	approvalId, visitorId, employeeId, status, requestedAt, decidedAt
meetingRooms	roomId, name, capacity, floor, status
roomBookings	bookingId, roomId, visitorId, startTime, endTime
visitorHistory	visitorId, checkInTime, checkOutTime, hostId, roomId, qrCode
notifications	notifId, userId, type, message, read, createdAt
securityLogs	logId, visitorId, guardId, action, timestamp
reports	reportId, type, generatedBy, dateRange, dataUrl
settings	key, value, updatedBy, updatedAt

9. API Request Flow

Every request passes through Express routing, authentication/role middleware, the controller, and the service layer before reaching Firestore, then returns a JSON response along the same path.

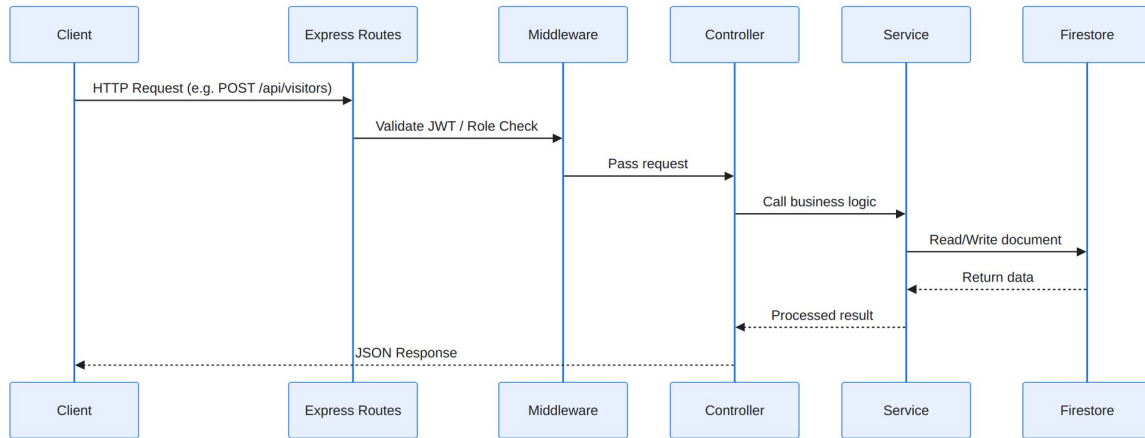


Figure 12 — Sequence diagram of a typical API request

Sample REST Endpoints

Method	Endpoint	Purpose	Role
POST	/api/auth/login	Login	All
POST	/api/visitors	Register visit	Visitor
GET	/api/approvals/pending	List pending approvals	Employee
PATCH	/api/approvals/:id	Approve / Reject	Employee
POST	/api/checkin/scan	Scan QR & check-in	Security
POST	/api/checkout/:visitorId	Check-out	Security
POST	/api/reception/walkin	Register walk-in visitor	Receptionist
GET	/api/reports/analytics	Fetch analytics	Admin
GET	/api/rooms/available	List free rooms	Receptionist / Employee

10. Visitor Lifecycle Diagram (State Machine)

The visitor record moves through a well-defined set of states from initial request through feedback, with a single rejection exit point from the approval stage.

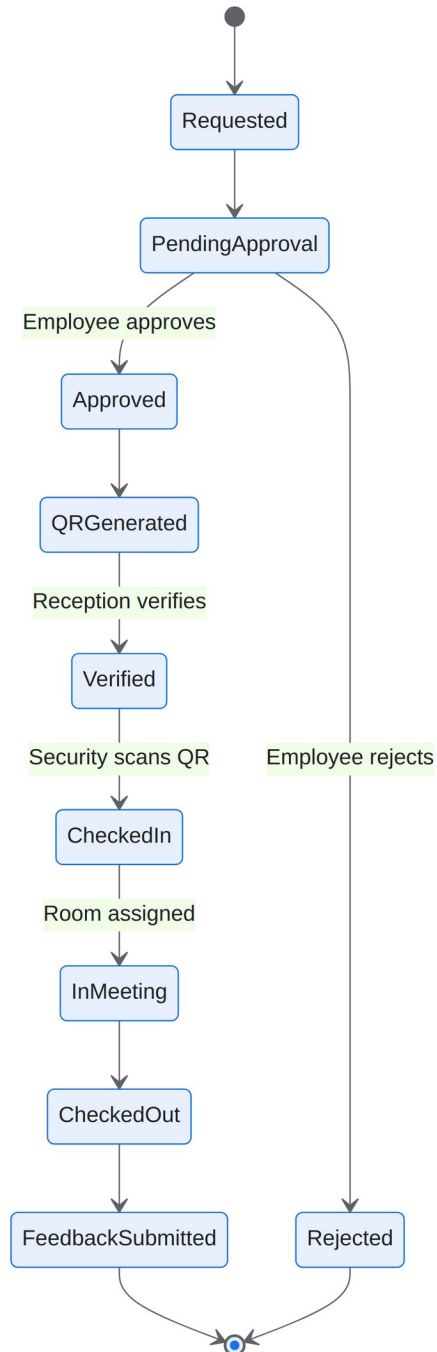


Figure 13 — Visitor lifecycle state machine

11. Dashboard Navigation Map

Once authenticated, each role lands on a dedicated dashboard exposing only the widgets and actions relevant to that role.

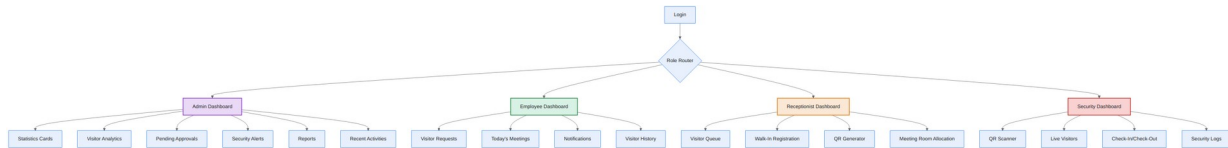


Figure 14 — Dashboard navigation map for all five roles

12. Design System Notes for Figma / FigJam Handoff

Color Coding

Blue = Visitor, Green = Employee, Orange = Receptionist, Red = Security, Purple = Admin, Yellow = Firebase / backend services.

Component Reuse

Build once and reuse across all five dashboards: Status Badge, QR Card, Dashboard Stat Card, Notification Toast, Approval Row.

Spacing & Grid

8px base spacing grid. 12-column responsive layout. Breakpoints at 1440 / 1024 / 768 / 375.

Typography Hierarchy

H1: 24-28px semibold (page titles) · H2: 18-20px medium (section headers) · Body: 14px regular · Caption: 12px (metadata/timestamps).

State Handling

Every flow node should have Figma variants for Default / Loading / Error / Empty states — especially Pending Approval, QR Scan, and Check-In steps.